

Sprawozdanie z Projektu

Honeypot SSH/Telnet

Autor: Mateusz Kantorski

Przedmiot: Bezpieczeństwo informatyczne

Data: 06.01.2026

1. Opis Architektury Systemu

System został zaprojektowany jako autorskie rozwiązanie typu Medium-Interaction Honeypot, łączące emulację usług SSH i Telnet w jednolitym ekosystemie monitorowania.

Główne komponenty systemu:

- **Rdzeń Asynchroniczny (Python 3.12 + AsyncIO):**

Całość logiki serwerowej oparta jest na asynchronicznej pętli zdarzeń. Zastosowanie `asyncio.BoundedSemaphore` pozwala na ścisłą kontrolę nad zasobami i limitowanie liczby jednoczesnych sesji (`MAX_CONCURRENT_SESSIONS`), co chroni hosta przed atakami typu DoS.

- **Moduł SSH (AsyncSSH):**

Zaawansowana emulacja oparta na bibliotece `asyncssh`. W przeciwieństwie do prostych skryptów, system obsługuje pełny handshake kryptograficzny, logowanie kluczem publicznym oraz autentykację hasłem.

- **Moduł Telnet :**

Implementuje niskopoziomą obsługę protokołu, w tym negocjację opcji (IAC) oraz autorski mechanizm buforowania linii tekstu z obsługą historii komend i klawiszy strzałek.

- **FakeShell:**

Komponent symulujący powłokę systemową Ubuntu 22.04. Przechwytuje on wejście użytkownika i generuje odpowiedzi na podstawie statycznej mapy komend, nie dając realnego dostępu do systemu plików kontenera.

- **AuthManager:** Singleton zarządzający stanem "wpuszczania" atakujących. Odpowiada za losowanie progów (Sticky Grant), liczenie unikalnych użytkowników na dane IP oraz detekcję skanowania agresywnego.

- **Warstwa Persystencji (SQLite):**

Wszystkie zdarzenia są zapisywane w czasie rzeczywistym do bazy danych SQLite. Baza składa się z dwóch głównych tabel:

- logs: Szczegółowy dziennik każdej interakcji.
- daily_summary: Agregacja statystyk (liczba ataków per protokół/dzień).

- **Warstwa Analityczna i Monitoring (Grafana & Prometheus):**

System został zintegrowany ze stosem monitoringu Prometheus, który pełni rolę bazy danych szeregów czasowych (Time-Series DB), oraz Grafaną jako warstwą wizualizacji. Dzięki temu możliwa jest analiza trendów ataku w czasie rzeczywistym.

Zaimplementowane dashboardy analityczne:

- **Velocity & Throughput (Dynamika Ataków):** Dashboard monitorujący natężenie zdarzeń na sekundę (\$EPS\$ – *Events Per Second*). Pozwala on na błyskawiczną identyfikację zmasowanych ataków typu *distributed brute-force* oraz wykrywanie nagłych skoków aktywności botnetów.
- **Attack Distribution (Taksonomia Zagrożeń):** Wizualizacja procentowego rozkładu wykrytych zdarzeń na podstawie opracowanych heurystyk (np. stosunek MALWARE_DOWNLOAD do RECONNAISSANCE). Dashboard ten pozwala ocenić główny profil motywacyjny napastników (np. czy serwer jest częściej celem prób infekcji, czy skanowania podatności).
- **Live Session Monitor (Status Operacyjny):** Panel monitorujący bieżącą liczbę aktywnych sesji interaktywnych dla protokołów SSH i Telnet. Wykorzystuje on metryki typu gauge do pokazywania aktualnego obciążenia systemu oraz liczby intruzów znajdujących się wewnątrz "pułapki" (FakeShell).

2. Użyte Heurystyki i Reguły Klasyfikacji

System implementuje dwustopniową klasyfikację zdarzeń: na poziomie **próby logowania** oraz na poziomie **wykonywanych komend**.

A. Klasyfikacja Prób Logowania

Klasyfikacja odbywa się w czasie rzeczywistym przy użyciu wyrażeń regularnych (Regex):

- **EXPLOIT_ATTEMPT:** Wykrywane, gdy login lub hasło zawiera wzorce takie jak \${jndi:} (Log4Shell), SELECT/UNION (SQL Injection), czy próby Path Traversal (/etc/passwd).
- **CREDENTIAL_STUFFING:** Przypisywane, gdy jedno IP próbuje zalogować się na więcej niż 3 różne nazwy użytkownika w krótkim odstępie czasu.

- **BOT_HARVESTING:** Automatycznie nadawane, gdy czas między połączeniem a próbą logowania jest mniejszy niż 0.5s (charakterystyczne dla szybkich botów).
- **PUBKEY_GUESS:** Próba uwierzytelnienia przy użyciu klucza asymetrycznego SSH
- **PASSWORD_GUESS:** Standardowe próby logowania niespełniające pozostałych klasyfikacji.
- **HONEYPOT_GRANT:** Moment, w którym atakujący przekroczy losowo wygenerowany próg prób (konfigurowalny zakres).

B. Klasyfikacja Aktywności w Shellu

Każda komenda wpisana przez atakującego jest analizowana pod kątem intencji:

- **MALWARE_DOWNLOAD:** Wykrycie narzędzi takich jak wget, curl, tftp lub linków URL.
- **RECONNAISSANCE:** Użycie komend uname, whoami, id, ifconfig lub próby odczytu plików systemowych (/proc/cpuinfo).
- **CRITICAL_ACTIVITY:** Wykrycie prób eskalacji (np. chmod +x, sudo), uruchamiania koparek kryptowalut (xmrig, mine.sh) lub tworzenia Reverse Shell (bash -i, /dev/tcp/).
- **COMMAND_EXEC** (command_exec) Reguła: Każdy ciąg znaków, który nie spełnia innych reguł przesłany przez użytkownika po uzyskaniu autoryzacji (wewnątrz Fake Shell).

3. Wyniki Testów i Obserwacje

Podczas testów z użyciem narzędzi **Nmap** oraz **Hydra** zaobserwowano następujące zachowania:

1. **Detekcja Skanowania (Nmap):** Dzięki metodzie check_aggressive_scan w serwerze Telnet, skanowanie nmap -sV zostało poprawnie zidentyfikowane. System odnotował 6 połączeń w ciągu 10 sekund i zamiast logować każdą sesję, oznaczył IP jako SCANNING z ograniczeniem zapisu do DB (throttling).
2. **Skuteczność "Tarpitu":** Wprowadzenie asyncio.sleep (0.1 - 0.5s) po błędnym logowaniu w Telnetcie skutecznie spowolniło ataki Hydry, zwiększając realizm emulacji.
3. **Analiza Payloadów:** Testy z użyciem "złośliwych" haseł (np. zawierających \${jndi:ldap...}) potwierdziły, że silnik Regex poprawnie klasyfikuje je jako EXPLOIT_ATTEMPT.
4. **Interaktywność Telnetu:** Weryfikacja manualna wykazała, że autorska implementacja Telnet poprawnie obsługuje Backspace i Ctrl+C, co jest rzadkością w prostych honeypotach.

3. Analiza Wyników i Weryfikacja Założeń

W celu weryfikacji skuteczności systemu przeprowadzono testy obciążeniowe oraz symulacje ataków przy użyciu narzędzi *Nmap* oraz *Hydra*. Poniżej przedstawiono kluczowe wnioski z przeprowadzonych badań:

1. Odporność na skanowanie i stabilność (Reliability)

System wykazał pełną stabilność podczas intensywnego skanowania narzędziem *nmap -A -T4*. System poprawnie zidentyfikował profilowanie usług, klasyfikując je jako SCANNING. Mimo dużej liczby pakietów próbkujących (probes), serwer nie uległ awarii, a wewnętrzne logi kontenera precyzyjnie zarejestrowały próby nawiązania sesji, co potwierdza poprawną separację warstwy sieciowej od aplikacyjnej.

2. Skuteczność mechanizmów spowalniających (Tarpit & Anti-bruteforce)

Zastosowanie techniki **Tarpit** znacząco wpłynęło na wydajność ataków słownikowych. Narzędzie *Hydra* zostało skutecznie spowolnione, co zwiększyło realizm emulacji (udawanie obciążenia realnego systemu) i utrudniło napastnikowi szybkie odgadnięcie parametrów usługi. Większość tych interakcji została poprawnie sklasyfikowana jako BOT_HARVESTING.

3. Mechanizm "Sticky Grant" i retencja dostępu

Testy potwierdziły poprawność działania autorskiego mechanizmu **Sticky Grant**. Zastosowanie losowego progu (np. 5 prób) skutecznie "oszukuje" automaty, które po otrzymaniu statusu AUTH_GRANTED kończą fazę brute-force i przechodzą do fazy wykonywania komend. Dzięki zapamiętywaniu adresu IP w AuthManager, powracający napastnik jest wpuszczany natychmiastowo, co realistycznie symuluje trwałość sesji użytkownika.

4. Zarządzanie zasobami i wydajność

System poprawnie zarządza pamięcią RAM poprzez cykliczne czyszczenie starych rekordów (metoda `cleanup_old_data`). Dzięki temu dane o historii połączeń (`_conn_history`) i logach skanowania nie narastają w nieskończoność, co jest kluczowe dla długofalowej stabilności honeypota typu *low-medium interaction*. Zastosowanie semaforów (`BoundedSemaphore`) skutecznie ograniczyło liczbę jednoczesnych procesów, chroniąc system przed wyczerpaniem deskryptorów plików.

5. Obserwacje dotyczące uwierzytelniania asymetrycznego (SSH Pubkey)

Podczas testów zauważono, że masowe generowanie prób logowania kluczem publicznym jest znacznie trudniejsze i bardziej zasobożerne dla atakującego niż brute-force haseł. W efekcie, logi dotyczące PUBKEY_GUESS stanowią mniejszy procent ogółu zdarzeń, co odzwierciedla realne trendy w internecie – większość botnetów skupia się na prostych danych uwierzytelniających (login:password).

6. Weryfikacja wizualna (Grafana)

Dashboardsy w czasie rzeczywistym poprawnie odzwierciedlały stan systemu. Wykresy *Velocity* natychmiastowo sygnalizowały rozpoczęcie testów wydajnościowych, a licznik *Live Status* precyzyjnie pokazywał liczbę aktywnych sesji wewnątrz powłoki FakeShell, co potwierdza poprawną integrację z eksporterem metryk Prometheusa.

4. Przykładowe wpisy z Bazy Danych (10 zdarzeń)

ID	Protokół	Typ Zdarzenia	Klasyfikacja	Adres IP	Szczegóły (Raw Payload)	Komentarz / Analiza
9164	TELNET	connection_lost	scanning	10.84.12 5.178	Connection closed without auth	Wykrycie skanowania portów (brak próby logowania).
9170	SSH	auth_attempt	exploit_attempt	10.84.12 5.178	sselect:[hash]	Wykrycie wzorca SQL Injection w polu loginu.
9171	SSH	auth_granted	honeypot_grant	10.84.12 5.178	sselect:[hash]	Mechanizm "Sticky Grant" – wpuszczenie intruza do sesji.

9175	SSH	command	reconnaissance	10.84.12 5.178	uname	Próba rozpoznania wersji systemu operacyjnego.
9173	SSH	command	malware_download	10.84.12 5.178	curl sfsa	Wykrycie użycia narzędzia do pobierania zewnętrznych danych.
8643	SSH	auth_attempt_publickey	pubkey_guess	10.84.12 5.50	root:SHA256:+Uuqmu...	Próba logowania z użyciem klucza publicznego.
9161	SSH	auth_attempt	bot_harvesting	10.84.12 5.50	adminl:[hash]	Logowanie w czasie < 0.5s od połączenia (charakterystyczne dla bota).
8334	SSH	auth_attempt	password_guess	10.84.12 4.234	userl:[hash]	Standardowa próba odgadnięcia hasła (Brute Force).
9179	SSH	command	command_exec	10.84.12 5.178	exit	Prawidłowe zakończenie sesji przez użytkownika.
8626	SSH	COMMAND	reconnaissance	10.84.12 5.26	uname	Wykrycie próby identyfikacji systemu operacyjnego.